LAPORAN graph.py

Program ini adalah sistem navigasi kampus sederhana berbasis graph yang dibuat menggunakan bahasa Python. Tujuan program ini adalah untuk mengelola data gedung dan jalan antar gedung, lalu mencari jalur tercepat menggunakan algoritma Dijkstra.

Program juga memiliki fitur pencarian gedung, penjelajahan graph menggunakan BFS dan DFS, serta pengecekan gedung terisolasi.

A. Adjacency List

1. Inisialisasi Graph (__init__)

```
class Graph:
    def __init__(self):
        self.adj: Dict[str, Optional[EdgeNode]] = {}
        self.node_names: Dict[str, str] = {}
```

Ini adalah constructor untuk membuat objek graph baru yang kosong. Di dalamnya ada dua dictionary (kamus data):

- self.adj: Menyimpan daftar tetangga (adjacency list). Key-nya adalah ID gedung (misal: "A1"), dan value-nya adalah head dari Linked List yang berisi gedung-gedung yang terhubung dengannya.
- self.node_names: Berfungsi sebagai direktori nama. Menyimpan hubungan antara ID gedung dan nama asli gedung tersebut (misal: "A1" dipetakan ke "Gerbang Utama").

2. Menambahkan Gedung baru (add_node)

```
    def add_node(self, node_id: str, nama: str) -> None:
        if node_id not in self.adj:
            self.adj[node_id] = None
            self.node_names[node_id] = nama
```

Fungsi: Untuk mendaftarkan gedung/titik baru ke dalam peta kampus.
Cara Kerja: Sistem memeriksa apakah ID gedung (node_id) sudah terdaftar atau belum. Jika belum, gedung akan dimasukkan ke self.adj dengan nilai awal None (artinya gedung ini belum punya jalan ke mana pun), dan nama aslinya disimpan ke dalam self.node_names.
Analisis Big-O: Operasi ini berjalan dalam waktu O(1) karena hanya memasukkan data ke dalam dictionary.

3. Menambahkan Jalan/Rute antar Gedung

- Fungsi: Untuk membuat jalan dua arah yang menghubungkan gedung u dan gedung v dengan jarak/waktu tertentu (bobot).
- Cara Kerja:
  1. Pertama, sistem memastikan kedua gedung (u dan v) sudah terdaftar di peta.
  2. Karena ini Linked List, jalan baru disisipkan di posisi paling depan (head). EdgeNode(v, bobot) dibuat, lalu pointer next-nya diarahkan ke tetangga lama dari u. Setelah itu, self.adj[u] diperbarui ke node baru ini.
  3. Langkah yang sama diulangi secara terbalik (dari v ke u) karena jalan kampus bisa dilewati bolak-balik (dua arah/undirected).

  Analisis Big-O: Kompleksitas waktunya adalah O(1) karena proses insert di bagian head pada Linked List tidak membutuhkan pergeseran data.

```python
def add_edge(self, u: str, v: str, bobot: int) -> None:
    if u in self.adj and v in self.adj:
        nv = EdgeNode(v, bobot)
        nv.next = self.adj[u]
        self.adj[u] = nv

        nu = EdgeNode(u, bobot)
        nu.next = self.adj[v]
        self.adj[v] = nu
```

  4. Mengambil daftar Tetangga Terdekat (Neighbors)

```python
def neighbors(self, u: str) -> List[Tuple[str, int]]:
    res = []
    curr = self.adj.get(u)
    while curr:
        res.append((curr.dest, curr.bobot))
        curr = curr.next
    return res
```

- Fungsi: Untuk melihat gedung apa saja yang terhubung langsung dengan gedung u, lengkap dengan jaraknya (bobot). Fungsi inilah yang nantinya akan sering dipanggil oleh algoritma Dijkstra, BFS, dan DFS untuk menelusuri jalur.
- Cara Kerja: Sistem mengambil head dari Linked List milik gedung u lewat self.adj.get(u). Menggunakan perulangan while, sistem berjalan menelusuri Linked List tersebut dari depan sampai ujung (None), lalu memasukkan ID tujuan (dest) dan jaraknya (bobot) ke dalam list res dalam bentuk Tuple.
- Analisis Big-O: Kompleksitas waktunya adalah O(derajat u) atau sebanding dengan jumlah jalan yang terhubung langsung ke gedung u.

B. BFS

1. Inisialisasi Variabel awal

```
def bfs(graph: Graph, source: str) -> List[str]:
    visited = set([source])
    result = []
    queue = QueueLL()
```

Fungsi: Menyiapkan struktur data pendukung sebelum penelusuran graf dimulai.

- Cara Kerja: * visited: Sebuah Set untuk mencatat gedung mana saja yang sudah pernah dikunjungi agar program tidak terjebak dalam perulangan tanpa akhir (infinite loop). Gedung awal (source) langsung dimasukkan ke sini.
- result: Sebuah List kosong yang nantinya akan menyimpan urutan gedung hasil penelusuran BFS.
- queue: Membuat objek antrean baru menggunakan QueueLL (Queue berbasis Linked List). BFS wajib menggunakan Queue karena menganut prinsip FIFO (First In, First Out) untuk mendata gedung secara berurutan per level/lapisan rute.

2. Memasukkan Gedung Awal ke Antrean

```
if source in graph.adj:
    queue.enqueue(source)
```

- Fungsi: Memulai proses antrean dari titik awal (misalnya dari Gerbang Utama).
- Cara Kerja: Sistem memastikan terlebih dahulu apakah gedung awal (source) terdaftar di dalam peta (graph.adj). Jika ada, gedung tersebut dimasukkan (enqueue) ke dalam antrean queue

3. Perulangan Utama Penelusuran (while loop)

```
while not queue.is_empty():
    u = queue.dequeue()
    result.append(u)
```

- Fungsi: Memproses satu per satu gedung yang ada di dalam antrean sampai antrean tersebut benar-benar kosong.
- Cara Kerja: Selama antrean tidak kosong, program akan mengeluarkan (dequeue) gedung yang berada di barisan paling depan dan menyimpannya di variabel u. Gedung u ini kemudian dimasukkan ke dalam list result sebagai tanda bahwa gedung tersebut resmi selesai diproses.

4. Pemeriksaan Tetangga Gedung (for loop di dalam while)

- Fungsi: Mengunjungi semua gedung tetangga yang terhubung langsung dengan gedung u.

- Cara Kerja: Program memanggil fungsi graph.neighbors(u) yang sudah kita bahas sebelumnya untuk mengambil semua gedung tetangga (v). Karakter garis bawah _ digunakan karena bobot/jarak jalan diabaikan dalam penelusuran BFS murni ini.
- Setiap tetangga v dicek; jika v belum pernah dikunjungi (not in visited), maka v akan ditandai sebagai gedung yang sudah dikunjungi (visited.add(v)) dan dimasukkan ke daftar antrean (queue.enqueue(v)) untuk diproses pada giliran berikutnya.

```
    for v, _ in graph.neighbors(u):
        if v not in visited:
            visited.add(v)
            queue.enqueue(v)
```

5. Mengembalikan Hasil Penelusuran

```
return result
```

- Fungsi: Mengirimkan output akhir dari fungsi BFS.
- Cara Kerja: Setelah semua gedung yang saling terhubung selesai dijelajahi dan antrean kosong, fungsi akan mengembalikan list result yang berisi urutan gedung-gedung yang berhasil dijangkau dari titik awal.

C. DFS

1. Inisialisasi Variabel Awal

```
def dfs(graph: Graph, source: str) -> List[str]:
    visited = set()
    result = []
    stack = StackLL()
```

- Fungsi: Menyiapkan wadah data yang diperlukan untuk pelacakan rute sebelum penelusuran dimulai.
- Cara Kerja: visited: Sebuah Set kosong untuk mencatat gedung mana saja yang sudah resmi dikunjungi agar program tidak memproses gedung yang sama berulang kali.
- result: Sebuah List kosong untuk menampung urutan gedung hasil penelusuran DFS secara berurutan.
- stack: Membuat objek tumpukan baru menggunakan StackLL (Stack berbasis Linked List). DFS menggunakan Stack karena menganut prinsip LIFO (Last In, First Out). Prinsip ini krusial untuk menciptakan efek mendalami rute hingga ujung dan melakukan backtracking (mundur ke gedung terakhir) saat menemui jalan buntu.

2. Memasukkan Gedung Awal ke Tumpahan

```
if source in graph.adj:
    stack.push(source)
```

- Fungsi: Memulai titik awal penelusuran dari gedung yang ditentukan (source).
- Cara Kerja: Sistem melakukan validasi apakah gedung awal terdaftar di dalam graf. Jika terdaftar, gedung tersebut dimasukkan ke dalam tumpukan menggunakan fungsi .push().

3. Perulanngan Utama Penelusuran (while loop)

```
while not stack.is_empty():
    u = stack.pop()
```

- Fungsi: Memproses tumpukan gedung satu per satu sampai tidak ada lagi gedung yang tersisa untuk dieksplorasi.
- Cara Kerja: Selama tumpukan stack belum kosong, program akan mengambil (pop) gedung yang berada di posisi paling atas/terakhir dimasukkan dan menyimpannya di variabel u.

4. Validasi Status Kunjungan

```
if u not in visited:
    visited.add(u)
    result.append(u)
```

- Fungsi: Memastikan gedung yang baru dikeluarkan dari stack benar-benar belum pernah diproses sebelumnya.
- Cara Kerja: Karena di dalam algoritma DFS berbasis iteratif (menggunakan stack manual) sebuah gedung bisa dimasukkan ke dalam stack lebih dari sekali dari tetangga yang berbeda, maka pengecekan if u not in visited sangat penting. Jika gedung u memang baru pertama kali dikunjungi, sistem akan memasukkannya ke set visited dan menambahkannya ke daftar hasil result.

5. Eksplorasi Tetangga Mendalam (for loop)

```
for v, _ in graph.neighbors(u):
    if v not in visited:
        stack.push(v)
```

- Fungsi: Memasukkan semua gedung tetangga yang terhubung langsung dengan gedung u ke dalam tumpukan untuk dijelajahi lebih dalam.

- Cara Kerja: Program memeriksa satu per satu gedung tetangga (v) dari gedung u. Jika tetangga tersebut belum pernah dikunjungi (not in visited), gedung v akan dimasukkan ke dalam tumpukan lewat perintah stack.push(v). Karena sifat Stack (LIFO), tetangga yang dimasukkan paling terakhir otomatis akan langsung diproses/ditelusuri paling pertama pada perulangan berikutnya, menciptakan efek penelusuran yang mendalam.

6. Mengembalikan Hasil Akhir

```
return result
```

- Fungsi: Mengirimkan output urutan rute hasil penelusuran.
- Cara Kerja: Ketika semua jalur sudah dijelajahi hingga tuntas dan stack menjadi kosong, fungsi mengembalikan *list* result yang berisi urutan jalur patroli mendalam yang telah dilewati.

D. Djikstra

1. Inisialisasi Jarak & Parent

```
def dijkstra(graph: Graph, source: str) -> Tuple[Dict[str, int], Dict[str, Optional[str]]]:
    INF = float('inf')
    dist = {v: INF for v in graph.adj}
    parent = {v: None for v in graph.adj}
    visited = set()
```

Fungsi: Mengatur jarak awal semua gedung menjadi tak hingga (INF), parent (asal usul rute) menjadi None, dan membuat daftar visited yang masih kosong.

2. Validasi & Titik Awal

```
    if source not in graph.adj:
        return dist, parent

    dist[source] = 0
```

Fungsi: Memastikan gedung awal ada di peta, lalu mengatur jarak dari gedung awal ke dirinya sendiri adalah 0.

3. Mencari Gedung dengan jarak Terkecil

```
    for v in graph.adj:
        if v not in visited and dist[v] < min_dist:
            min_dist = dist[v]
            u = v
```

Fungsi: Program memindai peta untuk mencari gedung u yang belum dikunjungi dan memiliki jarak (dist) paling kecil saat itu. Jika tidak ada lagi, perulangan berhenti (break).

4. Relaksasi (Pembaruan Jarak Tetangga)

Fungsi: Memeriksa semua tetangga gedung u. Jika lewat gedung u ternyata menghasilkan jarak yang lebih pendek (new_dist < dist[v]), maka data jarak ke gedung tetangga tersebut diperbarui, dan parent-nya diarahkan ke u.

Output: Di akhir, fungsi mengembalikan tabel jarak akhir (dist) dan peta rute (parent).

```
for v, weight in graph.neighbors(u):
    if v not in visited:
        new_dist = dist[u] + weight
        if new_dist < dist[v]:
            dist[v] = new_dist
            parent[v] = u
```

5. Menyusun Rute Jalan (reconstruct_path)

```
path = []
curr = target
```

Fungsi: Menyiapkan *list* rute kosong dan memulai pelacakan dari gedung tujuan (target).

6. Perulangan Mundur

```
while curr is not None:
    path.append(curr)
    if curr == source: break
    curr = parent.get(curr)
```

Fungsi: Program berjalan mundur dari gedung tujuan ke gedung asal menggunakan data parent. Setiap gedung dimasukkan ke *list* path. Jika sudah sampai gedung awal (source), pelacakan berhenti.

7. Membalikkan Hasil Rute

```
return path[::-1] if path and path[-1] == source else []
```

Fungsi: Karena pelacakan dilakukan terbalik (dari tujuan ke awal), perintah path[-1] digunakan untuk membalikkan urutan agar susunan rutenya benar (dari awal ke tujuan). Jika rute tidak tersambung, fungsi mengembalikan *list* kosong [].